Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
—
Институт компьютерных наук и кибербезопасности
Высшая школа кибербезопасности

ЛАБОРАТОРНАЯ РАБОТА № 1

«ОСНОВЫ РАБОТЫ В СРЕДЕ ATMEL STUDIO. АРХИТЕКТУРА И СИСТЕМА КОМАНД МИКРОКОНТРОЛЛЕРА ATMEGA32»

по дисциплине «Аппаратные средства вычислительной техники»

Выполнил
студент гр. 5151003/40001 Корпусов Д.Н

<подпись>

Руководитель Макаров А.С

<подпись>

Санкт-Петербург
2025

Содержание

Цель работы 3
Ход работы 3-31
1. Теоретическая часть 3-10
1.1 Общее описание отладочной платы EasyAVR v7 3
1.2 Основные компоненты и интерфейсы платы 3-4
1.3 Архитектура ATMega32 4
1.4 Память микроконтроллера 5
1.5 Периферийные модули 5
1.6 Адресное пространство микроконтроллера 6
1.7 Регистр состояния SREG 6-7
1.8 Система команд ATMega32 7-8
1.9 Работа с многоразрядными данными 8
1.10 Формат файла .hex 9
1.11 Расчёт времени выполнения программного кода 9
1.12 Командный цикл процессора 10
2. Практическая часть 11-31
2.1 Исследование работы алгоритма 11-16
2.2 Изучение .lss-файла 16-27
2.3 Изучение .hex-файла 28
2.4 Порядок выполнения команды ADC R15, R16 28-30
2.5 Расшифровка команд и их аргументов 31
Ответы на контрольные вопросы 32-39
Вывод 40
Листинг «.asm-файл» 41-43

ЦЕЛЬ РАБОТЫ

Изучение основ программирования микроконтроллеров семейства Atmel ATmega в среде разработки Atmel Studio. Знакомство с системой команд микроконтроллера ATmega32.

ХОД РАБОТЫ

1. Теоретическая часть

1.1 Общее описание отладочной платы EasyAVR v7

Отладочная плата **EasyAVR v7** представляет собой универсальную платформу для разработки и тестирования программно-аппаратных решений на базе микроконтроллеров семейства **Atmel AVR**. Плата оснащена встроенным программатором **mikroProg**, поддерживающим перепрограммирование 65 различных моделей микроконтроллеров AVR. Программатор подключается к персональному компьютеру через интерфейс USB 2.0. Состояние программатора определяется тремя светодиодами: светодиод **«Link»** сигнализирует об установленной USB-связи с компьютером, светодиод **«Active»** горит во время выполнения программы микроконтроллером, светодиод **«Data»** загорается при передаче данных между программатором и программным обеспечением.

1.2 Основные компоненты и интерфейсы платы

В состав отладочной платы **EasyAVR v7** входят следующие компоненты и интерфейсы:

— **программатор mikroProg** с поддержкой 65 моделей микроконтроллеров AVR;

— **кварцевый резонатор** для обеспечения тактовой частоты микроконтроллера;

— **четыре 8-разрядных порта ввода-вывода** (**PORTA–PORTD**), причём каждый вывод снабжён светодиодным индикатором, кнопкой и тремя внешними разъёмами для подключения периферийных устройств;

— **интерфейс UART**, выведенный одновременно на разъёмы RS-232 и USB через преобразователь сигналов на базе микросхемы FT232RL;

— **микросхема EEPROM** ёмкостью 1024 байта, подключённая к микроконтроллеру через интерфейс I²C со скоростью передачи данных до 400 кГц;

— **четыре семисегментных индикатора** для отображения цифр и некоторых символов;

— **два потенциометра** для имитации аналоговых входных сигналов, подключаемых к любому из двенадцати выводов микроконтроллера, поддерживающих аналоговый ввод;

— **пьезоэлектрический динамик (Piezo Buzzer)** с резонансной частотой 3,8 кГц, позволяющий генерировать звуковые сигналы в диапазоне от 2 до 4 кГц посредством широтно-импульсной модуляции (ШИМ);

— **интерфейсы для подключения жидкокристаллических дисплеев** (символьных 2×16 и графических 128×64), сенсорных панелей, температурных датчиков;

— **разъёмы** для плат расширения с поддержкой интерфейсов GPS, GSM, microSD, инфракрасного порта и Wi-Fi.

1.3 Архитектура ATMega32

Микроконтроллер **ATmega32** является 8-разрядным устройством, построенным на основе усовершенствованной архитектуры **RISC** (Reduced Instruction Set Computer). Он реализует **гарвардскую архитектуру**, при которой **память программ** и **память данных** физически разделены и имеют независимые адресные пространства. Такая организация позволяет одновременно выбирать команду из памяти программ и данные из памяти данных, что повышает производительность системы. Ядро процессора содержит **32 регистра общего назначения (R0–R31)**, все из которых напрямую подключены к арифметико-логическому устройству **(АЛУ)**. Благодаря этому большинство из **131 команд** системы выполняется за один машинный такт. При максимальной тактовой частоте 16 МГц микроконтроллер обеспечивает производительность до 16 миллионов инструкций в секунду (MIPS).

1.4 Память микроконтроллера

Память микроконтроллера ATmega32 включает следующие компоненты:

— **флэш-память программ** объёмом **32 КБ** с поддержкой внутрисхемного программирования (ISP) и режимом одновременного чтения и записи (Read-While-Write);

— **статическое оперативное запоминающее устройство (SRAM)** объёмом 2 КБ для хранения временных данных и организации стека;

— **энергонезависимая память EEPROM** объёмом 1024 байта для хранения конфигурационных параметров и данных, сохраняющихся при отключении питания.

1.5 Периферийные модули

Микроконтроллер оснащён богатым набором периферийных модулей:

— **два 8-разрядных и один 16-разрядный таймеры-счётчики** с режимами сравнения, захвата и генерации ШИМ-сигналов;

— **программируемый сторожевой таймер (Watchdog)** с собственным генератором для автоматической перезагрузки системы при сбоях;

— **программируемый последовательный интерфейс USART** для асинхронной и синхронной связи;

— **синхронный последовательный интерфейс SPI** в режимах ведущего (Master) и ведомого (Slave);

— **двухпроводной последовательный интерфейс I²C (TWI)** для подключения периферийных устройств с низким энергопотреблением;

— **8-канальный 10-битный аналого-цифровой преобразователь (АЦП)** с программируемым коэффициентом усиления входного сигнала;

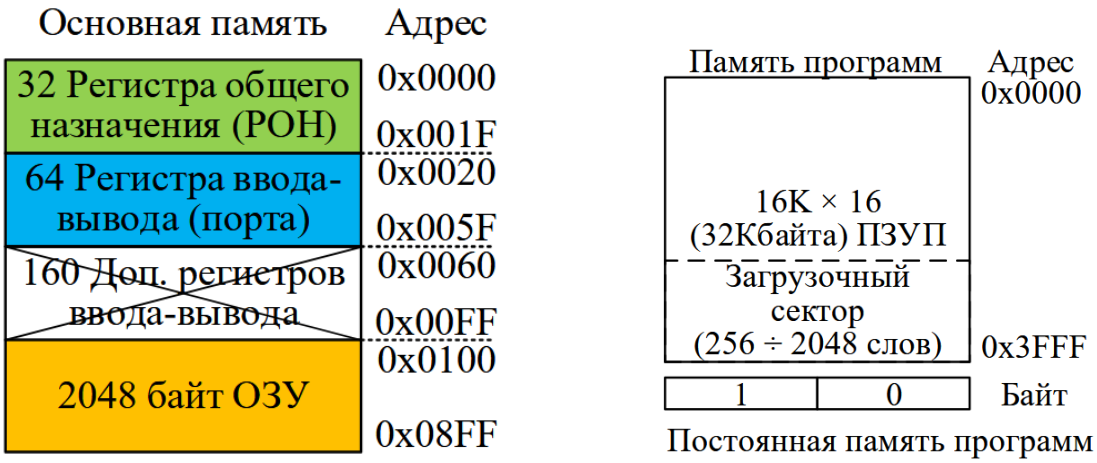
— **встроенный аналоговый компаратор** для сравнения аналоговых напряжений;

— четыре канала аппаратной генерации ШИМ-сигналов;

— интерфейс **JTAG** для аппаратной отладки и программирования.

1.6 Адресное пространство микроконтроллера

В микроконтроллере **ATmega32** используются два независимых адресных пространства. Первое пространство предназначено для **памяти данных** и включает **регистры общего назначения** (адреса 0x0000–0x001F), **регистры ввода-вывода** (0x0020–0x005F), **дополнительные регистры периферийных модулей** (0x0060–0x00FF) и **оперативную память SRAM (0x0100–0x08FF)**. Второе пространство отведено под **память программ** — 32 КБ флэш-памяти с адресацией от 0x0000 до 0x3FFF. Указатель стека формируется двумя 8-разрядными регистрами: **SPH** (старший байт) и **SPL** (младший байт). При инициализации программы указатель стека обычно устанавливается на конец области **SRAM** (адрес RAMEND), после чего стек растёт в сторону уменьшения адресов.



1.7 Регистр состояния SREG

Регистр состояния SREG (Status Register) является 8-разрядным регистром, содержащим флаги, отражающие результаты выполнения арифметических и логических операций. Назначение разрядов регистра следующее:

— **бит 0 (C, Carry flag)** — флаг переноса, устанавливается при возникновении переноса из старшего разряда результата операции;

— **бит 1 (Z, Zero flag)** — флаг нулевого результата, устанавливается, когда результат операции равен нулю;

— **бит 2 (N, Negative flag)** — флаг отрицательного результата, соответствует старшему (знаковому) биту результата;

— **бит 3 (V, Two's complement overflow)** — флаг переполнения при операциях со знаковыми числами;

— **бит 4 (S, Sign flag)** — бит знака, вычисляется как исключающее ИЛИ флагов N и V ($S = N \oplus V$) и корректно отражает знак результата даже при переполнении;

— **бит 5 (H, Half Carry flag)** — флаг половинного переноса, устанавливается при переносе из третьего разряда (используется в операциях двоично-десятичной арифметики);

— **бит 6 (T, Transfer bit)** — бит передачи, служит временным хранилищем для битовых операций и используется командами BST (Bit STore) и BLD (Bit LoaD);

— **бит 7 (I, Global Interrupt Enable)** — глобальный флаг разрешения прерываний; при значении 1 прерывания разрешены, при 0 — запрещены независимо от состояния регистров масок прерываний.

1.8 Система команд ATmega32

Система команд микроконтроллера включает 131 инструкцию, классифицируемую по функциональному назначению на следующие группы:

— **команды передачи управления:** безусловные (JMP, CALL, RET) и условные (например, BRNE — переход при неравенстве нулю, BREQ — переход при равенстве нулю);

— **команды передачи данных:** с прямой адресацией (OUT, IN), косвенной регистровой адресацией через регистровые пары X (R27:R26), Y (R29:R28) и Z (R31:R30), а также адресацией со смещением;

— **арифметические и логические команды:** бинарные операции (ADD, SUB, AND, OR), унарные операции (INC, DEC, CLR), операции с учётом флага переноса (ADC, SBC);

— **команды сдвига и циклического сдвига:** логические сдвиги (LSR, LSL), циклические сдвиги через флаг переноса (ROL, ROR);

— **битовые команды:** установка, сброс и проверка отдельных битов регистров и портов (например, SBR, CBR, SBRS, SBRC).

Для упрощения программирования на языке ассемблера в среде Atmel Studio предусмотрены директивы препроцессора: .def для переименования регистров (например, .def TMP = R20), .equ для определения именованных констант (например, .equ FREQ = 8000000), а также .macro для создания пользовательских макросов.

1.9 Работа с многоразрядными данными

Обработка многоразрядных данных в микроконтроллере **ATmega32** осуществляется двумя основными способами. Первый способ использует специализированные команды для работы с регистровыми парами: **ADIW** (сложение константы с парой регистров), **SBIW** (вычитание константы из пары регистров) и MOVW (**копирование пары регистров**). Эти команды имеют ограничения по номерам регистров — допустимы только пары с младшими номерами 24, 26, 28 и 30, а константы в командах ADIW и SBIW могут принимать значения от 0 до 63.

Второй способ основан на использовании флага переноса C и включает команды, учитывающие его состояние: **ADC** (сложение с учётом переноса), **SBC** (вычитание с учётом переноса), а также циклические сдвиги **ROL** и **ROR**. Например, сложение двух 32-разрядных чисел выполняется последовательно: сначала складываются младшие байты командой ADD, затем следующие байты — командой ADC, которая автоматически учитывает перенос из предыдущей операции. Аналогичным образом организуются сравнение (с использованием команды **CPC**) и арифметические сдвиги многобайтных значений. Для реализации циклического сдвига многобайтного числа применяется флаг передачи T: младший бит младшего байта сохраняется во флаг T командой BST, выполняется сдвиг всех байтов, после чего сохранённый бит переносится в старший разряд старшего байта командой BLD.

1.10 Формат файла .hex

Файл с расширением .hex представляет собой текстовое шестнадцатеричное представление двоичного образа программы, предназначенное для загрузки в микроконтроллер. Каждая строка файла (запись) начинается с символа двоеточия «:» и содержит шесть полей: длину данных (1 байт), смещение загрузки (2 байта), тип записи (1 байт), собственно данные (переменной длины) и контрольную сумму (1 байт). Тип записи «00» обозначает запись данных, «01» — конец файла, «02» — расширенный сегментный адрес. Контрольная сумма вычисляется как дополнение до двух суммы всех байтов записи (без учёта маркера «:»). Такой формат позволяет надёжно передавать программный код через последовательные интерфейсы и обеспечивает проверку целостности данных при загрузке в микроконтроллер.

1.11 Расчёт времени выполнения кода

Время выполнения программного фрагмента рассчитывается на основе количества тактов, необходимых для выполнения каждой команды. Например, команда LDI выполняется за 1 такт, команда DEC — также за 1 такт, а команда условного перехода BRNE требует 1 такт для проверки условия и дополнительно 1 такт при фактическом переходе (итого 2 такта при переходе и 1 такт при отсутствии перехода). Для вложенных циклов задержки общее количество тактов рассчитывается по формуле, учитывающей количество итераций внутреннего и внешнего циклов, а также время выполнения каждой команды. При тактовой частоте 8 МГц один такт длится 0,125 микросекунды, поэтому для получения задержки в 10 миллисекунд необходимо выполнить 80 000 тактов. Подбирая значения констант в регистрах-счётчиках и добавляя при необходимости команды NOP (каждая по 1 такту), можно точно настроить длительность задержки.

1.12 Командный цикл процессора.

При выполнении каждой команды процессор ATmega32 проходит через последовательность этапов, часть из которых обязательна для всех команд, а часть зависит от типа выполняемой операции. Обязательные этапы включают: передачу адреса текущей команды из счётчика команд в память программ, увеличение счётчика команд на единицу, извлечение кода команды из памяти и запись его в регистр команды, декодирование команды, передачу операндов в АЛУ, выполнение операции, обновление флагов в регистре SREG и запись результата в целевой регистр или порт. Для команд, работающих с оперативной памятью (например, LD, ST, PUSH, POP), дополнительно выполняются этапы чтения или записи данных из/в область SRAM. Для команд передачи управления (переходы, вызовы подпрограмм) новое значение записывается в счётчик команд, что изменяет последовательность выполнения программы. Такая организация командного цикла обеспечивает высокую производительность и предсказуемое время выполнения инструкций.

2. Практическая часть

2.1 Исследование алгоритма

В базовом примере был изменён фрагмент кода, выделенный зелёным, на фрагмент из своего варианта.

```
.def TMP = R20 ; Создаём псевдоним TMP для регистра R20
.org $000      ; Следующая команда в самом начале памяти команд
JMP reset      ; Указатель на начало программы

; Функция паузы. два вложенных цикла, которые просто сжигают такты:
delay:
LDI R30, 200; y = 200
LDI R29, 153; x = 153

delay_sub:
INC R29               ; x++ (инкремент) 1 такт
NOP                   ; 1 такт
BRNE delay_sub         ; «перейди по метке, если флаг Z = 0», т.е пока x!=0 (2 такта), если x=0 (1 такт)
NOP                ; 1 такт
DEC R30            ; y--
BRNE delay_sub      ; «перейди по метке, если флаг Z = 0», т.е пока y!=0 (2 такта), если y=0 (1 такт)
RET

; Начальная настройка
reset:

; настройка исходных значений
; LDI - LoaD Immediate (Загрузить непосредственно).
LDI TMP, 0x01    ; Загружаем шестнадцатеричное значение 0x01 (1 в десятичной) в TMP
MOV R0, TMP     ; Копируем значение из регистра TMP в регистр R0
```

```
CLR TMP          ; Устанавливает значение регистра TMP в 0
MOV R1, TMP      ; Копируем 0 из TMP в R1
MOV R2, TMP      ; Копирует 0 из TMP в R2
MOV R3, TMP      ; Копирует 0 из TMP в R3

; настройка портов ввода-вывода
; SER - (Set Register) Устанавливает все биты регистра в 1
SER TMP ; 0xFF
OUT DDRA, TMP ; Вывод (регистр направления данных (Data Direction Register))
OUT DDRB, TMP ; Вывод
OUT DDRC, TMP ; Вывод
OUT DDRD, TMP ; Вывод

; Запись 0xFF (все биты 1) все 8 контактов всех портов A, B, C, D настраиваются как выходы
; чтоб микроконтроллер мог выводить на них значения

; Установка вершины стека в конец ОЗУ
LDI TMP, HIGH(RAMEND) ; Старшие разряды адреса (8 бит максимального адреса)
OUT SPH, TMP          ; SPH - Stack Pointer High
LDI TMP, LOW(RAMEND)  ; Младшие разряды адреса
OUT SPL, TMP          ; SPL - Stack Pointer Low

; Основной цикл
loop:
; Циклический сдвиг 32-разрядного числа R0-R3
BST R0, 0 ; сохранение младшего бита во флаге T (т.е 1 у нас тут)
LSR R3 ; логический сдвиг вправо
ROR R2 ; циклический сдвиг вправо
ROR R1 ; циклический сдвиг вправо
ROR R0 ; циклический сдвиг вправо
BLD R3, 7 ; заполнение 7 бита значением из флага T
```

```
; Вывод 32-разрядного числа R0-R3 на порты PORTA-PORTD
OUT PORTA, R0
OUT PORTB, R1
OUT PORTC, R2
OUT PORTD, R3

; Пауза
CALL delay;

; Возврат в начало основного цикла
RJMP loop ;
```

Был создан новый проект в **Atmel Studio** на языке ассемблера, разместили исходный код в файл проекта. Был сформирован **«.lss»-файл** и **«.hex»-файл**. Был записан «.hex»-файл в контроллер с помощью программы **AVRFlash**.

Алгоритм работы текущей программы:

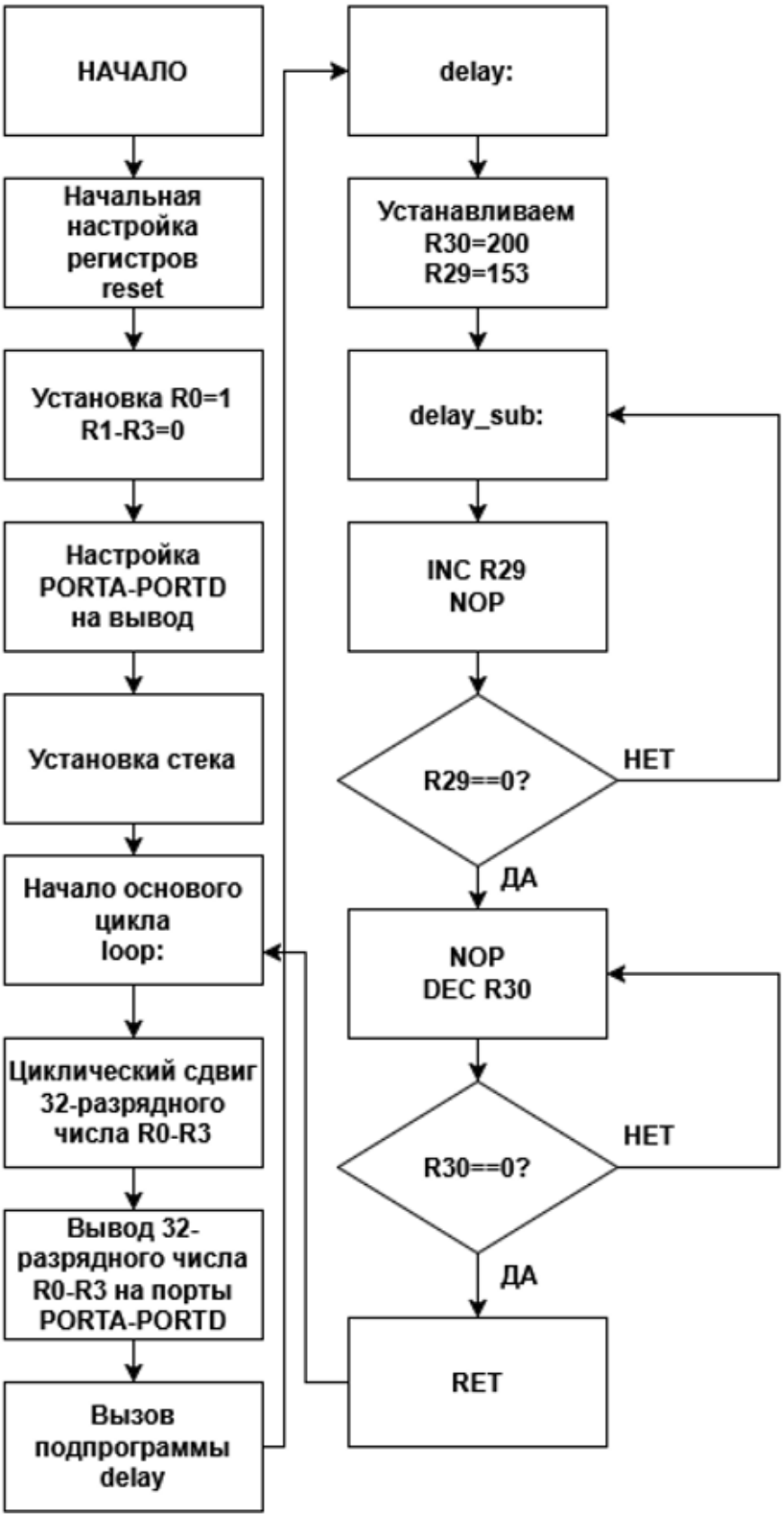


Рис. 1 «Блок-схема алгоритма»

В результате работы этого алгоритма наблюдатель видит один горящий светодиод, циклически перемещающийся по всем 32 светодиодам платы в направлении от PORTA.0 к PORTD.7. Скорость перемещения определяется подпрограммой задержки delay.c. Задержка составляет **204 789 тактов**.

Необходимо было рассчитать точное количество тактов процессора, требуемых для выполнения процедуры задержки delay, реализующей двойной вложенный цикл. Входные параметры: регистр R29 (переменная x) — счетчик внутреннего цикла; регистр R30 (переменная y) — счетчик внешнего цикла.

2 тактов – инициализация x и y значениями

+ (256 - x) * 2 тактов – количество инкрементов x и NOP в 1-ом проходе

+ 2 * (255 - x) + 1 тактов – количество переходов BRNE в 1-ом проходе

+ 2 * (y - 1) * 256 тактов – количество инкрементов x и NOP в оставшихся проходах

+ 2 * 255 * (y - 1) тактов – кол-во переходов BRNE(2 такта) для x в оставшихся проходах

+ (y - 1) - кол-во переходов BRNE(1 такт) для x в оставшихся проходах

+ 2 * y - количество декрементов x и NOP

+ 2 * (y - 1) + 1 – кол-во переходов BRNE для y

Всё сложили и упростили формулу:

T=1027y-4x+1

Получили зависимость количества тактов в delay от x и y. Видим, что зависимость получилась линейной.

Вычислили значения констант x и y при которых блок кода «delay» будет выполняться ровно 0,01 секунды (при тактовой частоте 8 МГц).

Нужно было решить уравнение: **1027y-4x+1=80000**

1027y-4x=79999

Взяли **y=78**, тогда 1027*78=80106

Осталось решить уравнение: **80106-4x=79999**

4x=107

X получается не целым числом, было принято решение добавить в конце функции delay дополнительную инструкцию **NOP (1 такт)**, в итоге получится уравнение:

4x=108

x=27

При x=27, y=78 мы получили задержку 10 мс при частоте процессора 8 МГц.

2.2 Изучение .lss файла

Был изучен сформированный .lss файл. Были выписаны адреса всех меток программы:

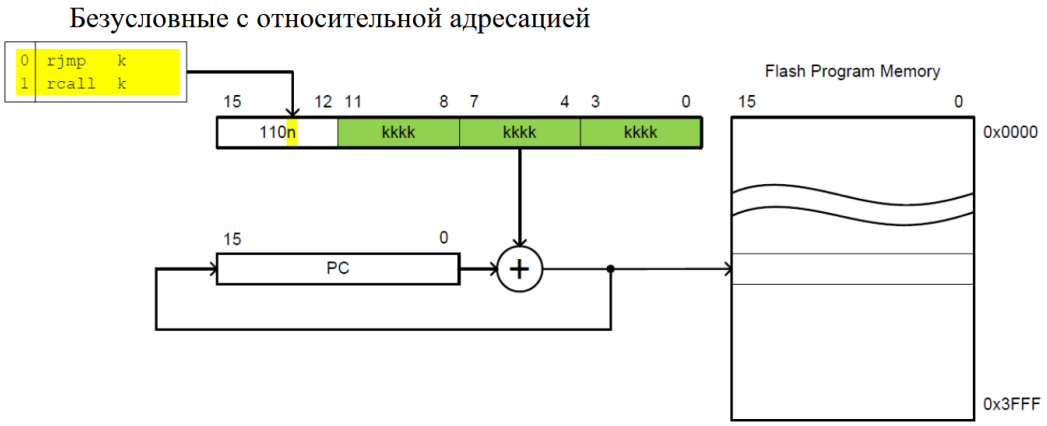
1) reset: 000b

```
000000 940c 000b                    JMP reset
```

2) delay: 0002

```
000024 940e 0002                    CALL delay;
```

3) **loop**: задаётся с помощью смещения ff3

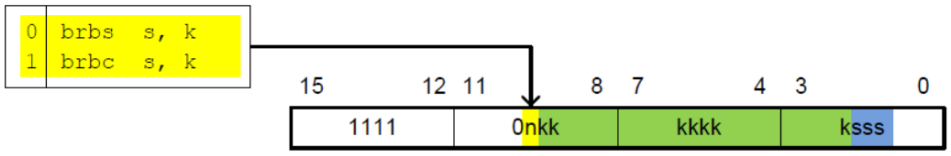



```
000026 cff3                         RJMP loop ;
```

c в hex: 1100 , а дальше идёт у нас адрес смещения **ff3**

4) delay_sub:

```
                                        delay_sub:
000004 95d3                             INC R29
000005 0000                             NOP
000006 f7e9                             BRNE delay_sub
```



alias			
SREG	sss	brbs s, k	brbc s, k
I	111	brie k	brid k
T	110	brts k	brtc k
H	101	brhs k	brhc k
S	100	brlt k	brge k
V	011	brvs k	brvc k
N	010	brmi k	brpl k
Z	001	breq k	brne k
C	000	brcs k brlo k	brcc k brsh k

brbs - Branch if Bit Set (флаг = 1)

brbc - Branch if Bit Clear (флаг = 0)

f739 – 1111 0111 1110 1001

if (Z = 0) then PC ← PC + k + 1

k= 1111101

PC(hex)=000006 (адрес текущей инструкции)

PC(binary)= 0000110

PC_new=0000110+1111101+1=

=0000011+1=0000100(binary)=000004(hex)

Используемые форматы команд в части состава и размерности операндов:

1) Команды пересылки:

NOP (Пустая команда), 16 бит (1 такт), операнды – нет

Алгоритм работы:

Процессор считывает 16-битный код команды из Flash-памяти по адресу, указанному в счетчике команд (PC), PC автоматически увеличивается на 1. Декодер команд распознает все биты команды **0000.0000.0000.0000** как команду NOP. Операнды отсутствуют. Никаких действий над регистрами, флагами статусного регистра или портами ввода-вывода не производится. Процессор просто завершает текущий машинный цикл, затрачивая ровно 1 такт времени. Содержимое всех регистров и флагов остается неизменным. Процессор готов к следующей команде по новому адресу PC.

LDI (Загрузка константы), 16 бит, 1 такт, операнды – Rd(регистр), k (константа – 8 бит: 1110.KKKK.dddd.KKKK , d-номер регистра)

Алгоритм работы:

Процессор считывает 16-битный код команды из Flash-памяти по адресу, указанному в счетчике команд (**PC**), PC автоматически увеличивается на 1. Декодер команд распознает старшие биты **1110** как команду **LDI**. Из младших битов команды извлекаются: **Номер целевого регистра** Rd (например, R30), **8-битная константа K** (например, 200 или 0xC8). Значение константы K передается напрямую во входную шину регистра Rd. Данные записываются в регистр Rd: Rd ← K. Команда выполнена за **1 такт**. Процессор готов к следующей команде по новому адресу PC.

MOV(Копирование регистров), 16 бит, 1 такт, операнды – Rd, Rr , 0010.11rd.dddd.rrrr , где d,r – номера регистров

Алгоритм работы:

Процессор считывает 16-битный код команды из Flash-памяти по адресу, указанному в счетчике команд (PC), PC автоматически увеличивается на 1. Декодер команд распознает старшие биты **0010.11** как команду MOV. Из младших битов команды извлекаются: Номер регистра-приемника Rd и номер регистра-источника Rr. Содержимое регистра Rr считывается и передается по внутренней шине данных на вход регистра Rd. Данные записываются в регистр Rd: Rd ← Rr. Флаги статусного регистра не изменяются. Команда выполнена за 1 такт. Процессор готов к следующей команде по новому адресу PC.

OUT(Запись значения регистра в порт), 16 бит, 1 такт, операнды – P(порт), Rr(номер регистра), 1011.1AAr.rrrr.AAAA

Алгоритм работы:

Процессор считывает 16-битный код команды из Flash-памяти по адресу, указанному в счетчике команд (PC), PC автоматически увеличивается на 1. Декодер команд распознает старшие биты **1011.1** как команду OUT. Из младших битов команды извлекаются: 6-битный адрес порта ввода-вывода P (обозначен как AAAAAA в коде, разнесен по разным частям инструкции) и номер исходного регистра Rr (обозначен как **r.rrrr**). Значение, находящееся в регистре Rr, передается по внутренней шине данных непосредственно в порт ввода-вывода с адресом P. Выполняется операция записи: P ← Rr. Флаги статусного регистра не изменяются. Команда выполнена за 1 такт. Процессор готов к следующей команде по новому адресу PC.

2) Команды вызова подпрограммы:

RET (Возврат из подпрограммы), размерность 16 бит (4 такта), операнды – нет

Алгоритм работы: Процессор считывает 16-битный код команды из Flash-памяти по адресу, указанному в счетчике команд (PC), PC автоматически увеличивается на 1. Декодер команд распознает биты

1001.0101.0000.1000 как команду RET. Операнды отсутствуют. Выполняется операция возврата из подпрограммы: из стека последовательно извлекаются два байта адреса возврата — сначала старший байт загружается в регистр PCh, затем младший байт в регистр PCl (PCh ← Stack, PCl ← Stack). Таким образом, счетчик команд PC восстанавливает значение, которое было сохранено при вызове подпрограммы, и управление передается на команду, следующую за инструкцией вызова. Команда выполняется за 4 такта. Процессор готов к выполнению следующей команды по восстановленному адресу PC.

CALL (Вызов подпрограммы), 32 бита, 4 такта, операнды – k(адрес), k∈[0;16384], 1001.010k.kkkk.111k.kkkk.kkkk.kkkk.kkkk,
Stack ← PCl, Stack ← PCh, PC ← k

Алгоритм работы: Процессор считывает 32-битный код команды (два последовательных слова) из Flash-памяти по адресу, указанному в счетчике команд (PC), PC автоматически увеличивается на 2. Декодер команд распознает старшие биты **1001.010** и специфические биты **111** в структуре команды как команду **CALL**. Из разбросанных по обоим словам команды битов извлекается полный **16-битный адрес перехода k** (например, адрес метки подпрограммы). Перед выполнением перехода процессор сохраняет адрес следующей команды (текущее значение PC) в стек: сначала в стек записывается младший байт счетчика команд (**Stack ← PCl**), затем старший байт (**Stack ← PCh**). После сохранения адреса возврата выполняется безусловный переход: счетчик команд загружает извлеченный адрес **k** (**PC ← k**). Команда выполняется за **4 такта**. Процессор начинает выполнение подпрограммы по новому адресу PC.

3) Арифметические команды:

INC (инкремент), 16 бит, 1 такт, операнды – Rd (регистр)

Алгоритм работы:

Процессор считывает 16-битный код команды из Flash-памяти по адресу, указанному в счетчике команд (PC), PC автоматически увеличивается на 1. Декодер команд распознает старшие биты **1001.010** и младшие биты **0011** как команду INC. Из средней части команды извлекается номер целевого регистра Rd (диапазон R0–R31). Текущее значение регистра Rd поступает в **арифметико-логическое устройство (АЛУ)**, где выполняется операция сложения с единицей: Rd ← Rd + 1. Полученный результат записывается обратно в тот же регистр Rd. Одновременно обновляются **флаги статусного регистра SREG**: **Z** (если результат равен 0), **N** (по значению старшего бита результата), **V** (при переполнении знакового числа) и **S** (как результат XOR флагов N и V). Флаг переноса **C** не изменяется. Команда выполнена за **1 такт**. Процессор готов к следующей команде по новому адресу PC.

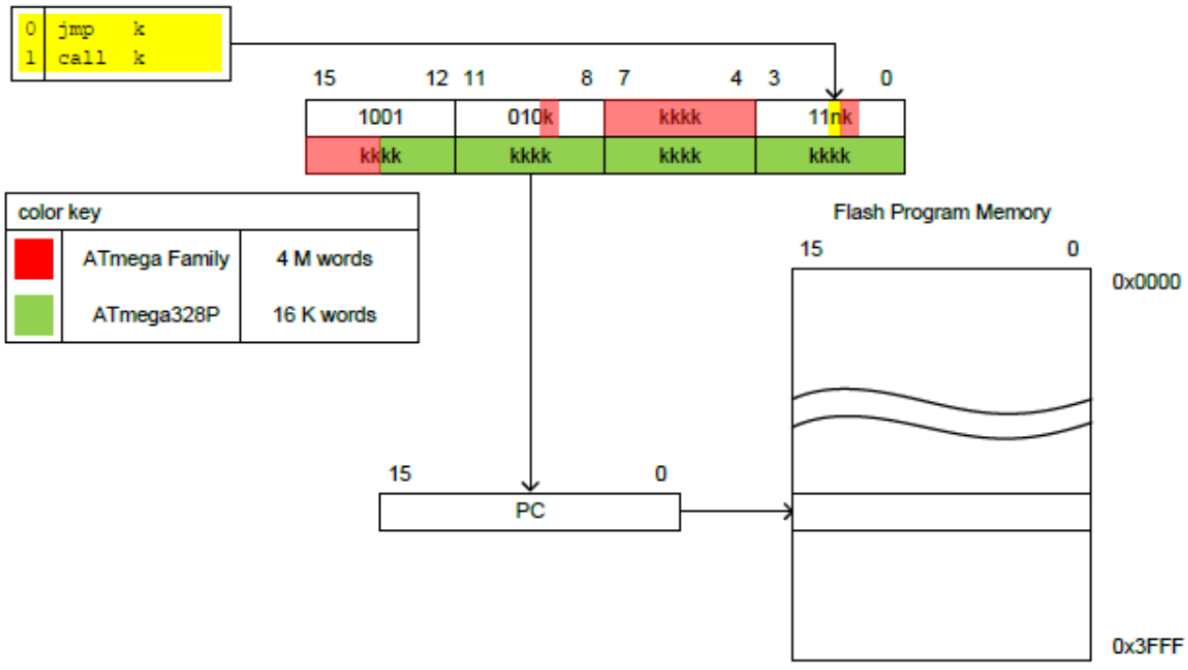
DEC (декремент), 16 бит, 1 такт, операнды – Rd (регистр)

Алгоритм работы: Процессор считывает 16-битный код команды из Flash-памяти по адресу, указанному в счетчике команд (PC), PC автоматически увеличивается на 1. Декодер команд распознает старшие биты **1001.010** и младшие биты **1010** как команду **DEC**. Из средней части команды извлекается **номер целевого регистра Rd** (диапазон R0–R31). Текущее значение регистра Rd поступает в арифметико-логическое устройство (АЛУ), где выполняется операция вычитания единицы: **Rd ← Rd - 1**. Полученный результат записывается обратно в тот же регистр Rd. Одновременно обновляются флаги статусного регистра SREG: Z (если результат равен 0), N (по значению старшего бита результата), V (при переполнении знакового числа, например, при переходе от -128 к 127) и S (как результат XOR флагов N и V). Флаг переноса C не изменяется. Команда выполнена за **1 такт**. Процессор готов к следующей команде по новому адресу PC.

4) Команды безусловного перехода:

JMP (прямой безусловный переход), 32 бита, 3 такта, операнды – k (адрес), k∈[0;16384]

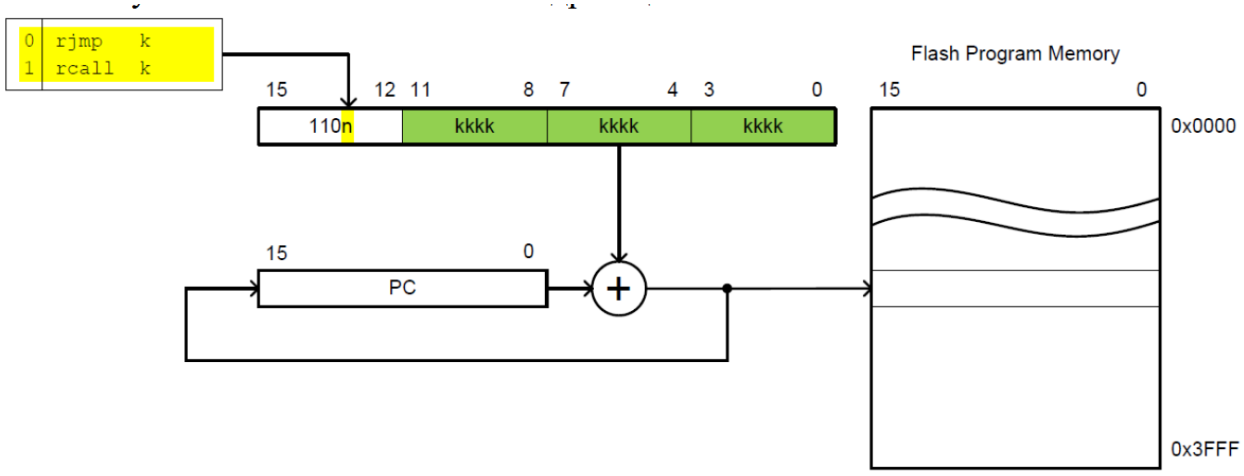
Алгоритм работы: Процессор считывает 32-битный код команды (два последовательных слова) из Flash-памяти по адресу, указанному в счетчике команд (PC), PC автоматически увеличивается на 2. Декодер команд распознает старшие биты **1001.010** и специфические биты **110** в структуре команды как команду **JMP**. Из разбросанных по обоим словам команды битов извлекается полный **16-битный адрес перехода k** (прямой адрес в пределах памяти программ). Выполняется безусловная загрузка этого адреса в счетчик команд: **PC ← k**. При этом содержимое стека не изменяется, и адрес возврата не сохраняется. Команда выполняется за **3 такта**. Процессор начинает выполнение программы по новому абсолютному адресу PC.



RJMP (Относительный безусловный переход), 16 бит (1100.kkkk.kkkk.kkkk), k∈[-2048;2047], 2 такта, PC ← PC + k + 1

Алгоритм работы: Процессор считывает 16-битный код команды из Flash-памяти по адресу, указанному в счетчике команд (PC), PC

автоматически увеличивается на 1. Декодер команд распознает старшие биты **1100** как команду **RJMP**. Из младших 12 бит команды извлекается **12-битное знаковое смещение k** (диапазон от -2048 до +2047), которое кодируется в дополнительном коде. Это смещение складывается с текущим значением счетчика команд (который уже был увеличен на единицу на этапе выборки) по формуле: **PC ← PC + k + 1**. Если смещение отрицательное, происходит переход назад в памяти; если положительное — вперед. Команда выполняется за **2 такта**. Процессор готов к выполнению следующей команды по новому вычисленному адресу PC.

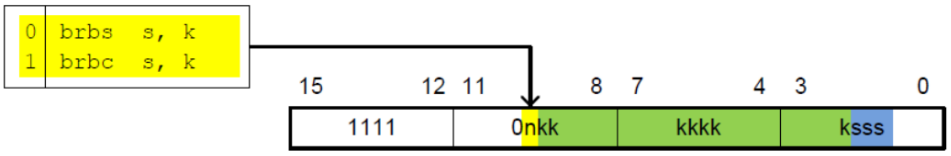


4) Команды условного перехода:

BRNE (Переход если неравно (Z=0)), if (Z = 0) then PC ← PC + k + 1, 16 бит, 1 такт(Z=0) или 2 такта(Z!=0), 1111.01kk.kkkk.k001, k – смещение, **k∈[-64;63].**

Алгоритм работы: Процессор считывает 16-битный код команды из Flash-памяти по адресу, указанному в счетчике команд (PC), PC автоматически увеличивается на 1. Декодер команд распознает старшие биты **1111.01** и младшие биты **001** как команду **BRNE** (алиас для BRBC 1 — переход, если бит сброшен). Из разбросанных по команде битов извлекается **7-битное знаковое смещение k** (диапазон от -64 до +63), закодированное в дополнительном коде. Процессор проверяет состояние флага **Z** (Zero) в

регистре статусов SREG. Если **Z = 0** (результат предыдущей операции не равен нулю), выполняется переход: к текущему значению PC прибавляется извлеченное смещение **k** по формуле **PC ← PC + k + 1**. Если **Z = 1**, переход не выполняется, и программа продолжается со следующей команды. Команда выполняется за **1 такт** (если перехода нет) или **2 такта** (если переход выполнен). Процессор готов к следующей команде по новому (или следующему последовательному) адресу PC.



alias			
SREG	sss	brbs s, k	brbc s, k
I	111	brie k	brid k
T	110	brts k	brtc k
H	101	brhs k	brhc k
S	100	brlt k	brge k
V	011	brvs k	brvc k
N	010	brmi k	brpl k
Z	001	breq k	brne k
C	000	brcs k brlo k	brcc k brsh k

5) Битовые команды:

CLR (Очистка всех битов регистра), 16 бит, 1 такт, операнды – Rd, операция - Rd ← Rd ⊕ Rd

Алгоритм работы: Процессор считывает 16-битный код команды из Flash-памяти по адресу, указанному в счетчике команд (PC), PC автоматически увеличивается на 1. Декодер команд распознает биты **0010.01dd.dddd.dddd** как команду **CLR**. Из средней части команды извлекается **номер целевого регистра Rd** (диапазон R0–R31). Содержимое регистра Rd поступает в арифметико-логическое устройство (АЛУ), где выполняется операция побитового сложения по модулю 2 самого числа с собой: **Rd ← Rd ⊕ Rd**. Поскольку любой бит, сложенный сам с собой, дает 0, результат операции всегда равен нулю (**Rd ← $00**). Одновременно

обновляется флаг **Z** (устанавливается в 1, так как результат равен нулю), а флаги **N**, **V** и **S** сбрасываются в 0. Команда выполнена за **1 такт**. Процессор готов к следующей команде по новому адресу PC.

SER (Установка всех битов регистра), 16 бит, 1 такт, операнды – Rd, 1110.1111.dddd.1111, d-номер регистра ∈[16;31], операция Rd ← $FF

Алгоритм работы: Процессор считывает 16-битный код команды из Flash-памяти по адресу, указанному в счетчике команд (PC), PC автоматически увеличивается на 1. Декодер команд распознает старшие биты **1110.1111** и младшие биты **1111** как команду **SER**. Из средней части команды извлекается **номер целевого регистра Rd** (доступны только регистры старшей половины: R16–R31). Внутренняя схема процессора формирует константу **$FF** (все восемь битов равны единице: 11111111) и записывает её непосредственно в регистр Rd: **Rd ← $FF**. Эта команда эквивалентна загрузке константы 255 (LDI Rd, 0xFF), но выполняется специальным кодом. Флаги статусного регистра не изменяются. Команда выполнена за **1 такт**. Процессор готов к следующей команде по новому адресу PC.

6) Команды изменения признаков:

BST (Сохранить бит регистра Rr во флаг T регистра SREG), 16 бит, 1 такт, операнды – Rr, b(номер бита)

Алгоритм работы: Процессор считывает 16-битный код команды из Flash-памяти по адресу, указанному в счетчике команд (PC), PC автоматически увеличивается на 1. Декодер команд распознает старшие биты **1111.101** как команду **BST**. Из младших битов команды извлекаются: **номер исходного регистра Rr** и **номер бита b** (от 0 до 7) внутри этого регистра. Процессор обращается к регистру Rr, считывает значение указанного бита **b** и копирует его во флаг **T** (Bit Transfer Storage) статусного регистра SREG: **T ← Rr(b)**. Остальные биты регистра Rr и остальные флаги SREG остаются

неизменными. Команда выполнена за **1 такт**. Процессор готов к следующей команде по новому адресу PC.

BLD (Считать флаг T регистра SREG в бит регистра Rd), 16 бит, 1 такт, операнды – Rd, b

Алгоритм работы: Процессор считывает 16-битный код команды из Flash-памяти по адресу, указанному в счетчике команд (PC), PC автоматически увеличивается на 1. Декодер команд распознает старшие биты **1111.100** как команду **BLD**. Из младших битов команды извлекаются: **номер целевого регистра Rd** и **номер бита b** (от 0 до 7) внутри этого регистра. Процессор считывает текущее значение флага **T** (Bit Transfer Storage) из статусного регистра SREG и записывает его в указанный бит **b** регистра Rd: **Rd(b) ← T**. Все остальные биты регистра Rd остаются неизменными, как и остальные флаги SREG. Команда выполнена за **1 такт**. Процессор готов к следующей команде по новому адресу PC.

7) Команды сдвига:

LSR (Логический сдвиг вправо), 16 бит, 1 такт, операнды – Rr,
Rd(n) ← Rd(n+1), Rd(7) ← 0, C←Rd(0)

Алгоритм работы:

Процессор считывает 16-битный код команды из Flash-памяти по адресу, указанному в счетчике команд (PC), PC автоматически увеличивается на 1. Декодер команд распознает старшие биты **1001.010** и младшие биты **0110** как команду **LSR**. Из средней части команды извлекается **номер целевого регистра Rd** (диапазон R0–R31). Содержимое регистра Rd поступает в арифметико-логическое устройство (АЛУ), где выполняется операция логического сдвига всех битов вправо на одну позицию: каждый бит с номером n получает значение бита n+1 (**Rd(n) ← Rd(n+1)**). Старший бит (бит 7) принудительно устанавливается в ноль (**Rd(7) ← 0**), что эквивалентно делению беззнакового числа на 2. Младший бит (бит 0),

который был вытеснен при сдвиге, записывается во флаг переноса **C** регистра SREG (**C ← Rd(0)**). Одновременно обновляются флаги **Z** (устанавливается, если результат равен 0), **N** (сбрасывается в 0, так как старший бит всегда 0) и **V** (результат XOR флагов N и C). Команда выполнена за **1 такт**. Процессор готов к следующей команде по новому адресу PC.

ROR (Циклический сдвиг вправо через флаг переноса), 16 бит, 1 такт, операнды – Rd, Rd(7)←C, Rd(n)← Rd(n+1), C←Rd(0)

Алгоритм работы: Процессор считывает 16-битный код команды из Flash-памяти по адресу, указанному в счетчике команд (PC), PC автоматически увеличивается на 1. Декодер команд распознает старшие биты **1001.010** и младшие биты **0110** как команду **LSR**. Из средней части команды извлекается **номер целевого регистра Rd** (диапазон R0–R31). Содержимое регистра Rd поступает в арифметико-логическое устройство (АЛУ), где выполняется операция логического сдвига всех битов вправо на одну позицию: каждый бит с номером n получает значение бита n+1 (**Rd(n) ← Rd(n+1)**). Старший бит (бит 7) принудительно устанавливается в ноль (**Rd(7) ← 0**), что эквивалентно делению беззнакового числа на 2. Младший бит (бит 0), который был вытеснен при сдвиге, записывается во флаг переноса **C** регистра SREG (**C ← Rd(0)**). Одновременно обновляются флаги **Z** (устанавливается, если результат равен 0), **N** (сбрасывается в 0, так как старший бит всегда 0) и **V** (результат XOR флагов N и C). Команда выполнена за **1 такт**. Процессор готов к следующей команде по новому адресу PC.

2.3 Изучение сформированного «.hex» файла

Была изучена структура сформированного «.hex»-файла, было определено количество записей в файле и количество машинных слов программы.

```
:020000020000FC
:100000000C940B00E8ECD9E9D3950000E9F7000067
:10001000EA95D1F7089541E0042E4427142E242EAA
:10002000342E4FEF4ABB47BB44BB41BB48E04EBFF9
:100030004FE54DBF00FA36942794179437F88C
:0E0040000BBA18BA25BA32BA0E940200F3CFEA
:00000001FF
```

Файл прошивки выполнен в формате Intel HEX и содержит 7 записей. Каждая запись начинается с символа : и включает: длину данных (**LL**), адрес загрузки (**AAAA**), тип записи (**TT**), данные (**DD...**), контрольную сумму (**CC**).

После анализа установлено:

Общее количество записей: 7

Количество записей с данными программы (тип 00): 5

Общий объём кода программы: 78 байт

Количество машинных слов программы: 39 (так как 1 машинное слово AVR = 2 байта; 78 / 2 = 39)

Тип 02 (Extended Segment Address) — это первая строка :020000020000FC.

Тип 00 (Data) — основные строки с кодом.

Тип 01 (End of File) — последняя строка.

2.4 Порядок выполнения команды ADC R15, R16 внутри ЦП ATmega32

Машинный код команды: **0001.11rd.dddd.rrrr** Для регистров R15 (01111) и R16 (10000) код будет: **0001.1110.1111.0000** (в HEX: **1D F0**).

1) Выборка команды (Fetch)

Действие: Содержимое счетчика команд (СчК) (текущий адрес программы) передается по шине адреса в Память программ (ПЗУП).

Управляющие сигналы: Формируется активный сигнал **MEM_READ** (чтение ПЗУП).

Пересылаемые данные: Из ПЗУП по шине данных считывается 16-битное слово команды (1DF0) и записывается в **Регистр команды**.

Параллельное действие: Одновременно устройство управления (УУ) формирует сигнал **INC_PC**, увеличивая значение СчК на единицу (**СчК ← СчК + 1**) для подготовки к следующей инструкции.

2) Декодирование команды (Decode)

Действие: Биты из регистра команды поступают в декодер команд.

Анализ: Декодер распознает старшие биты 0001.11 как операцию **ADC** (сложение с переносом).

Извлечение операндов: Из соответствующих полей регистра команды выделяются номера регистров:

Поле ddddd = 01111 → номер регистра-приемника **R15**.

Поле rrrrr = 10000 → номер регистра-источника **R16**.

Формирование сигналов: УУ генерирует пакет управляющих сигналов:

SEL_RD=15 и **SEL_RR=16** — адреса для выбора ячеек в регистровом файле.

READ_REG — разрешение на чтение данных из выбранных регистров.

OP_ADC — установка режима «Сложение с переносом» в АЛУ.

READ_FLAG_C — сигнал для считывания флага переноса.

3) Извлечение операндов (Read Operands)

Действие: Под действием сигналов SEL_RD, SEL_RR и READ_REG происходит одновременное чтение данных из регистрового файла **(РОН)**:

Данные из ячейки **R15** передаются на вход **А** АЛУ.

Данные из ячейки **R16** передаются на вход **Б** АЛУ.

Чтение флага: Одновременно по сигналу READ_FLAG_C текущее значение бита **C** (Carry, бит 0) из регистра флагов (SREG) считывается и подается на третий вход сумматора АЛУ (Carry-in).

Итог этапа: На входах сумматора сформированы три слагаемых: R15, R16 и C.

4) Выполнение операции (Execute)

Действие: В **Арифметико-Логическом Устройстве (АЛУ)** под управлением сигнала OP_ADC выполняется арифметическое сложение: SUM $= A + B + C_{in} = R15 + R16 + C$

Формирование флагов: Логическая схема статусов анализирует результат суммы и мгновенно формирует новые состояния для флагов:

SET_Z (если результат = 0).

SET_C (если произошел перенос из 7-го разряда).

SET_N (копируется 7-й бит результата).

SET_V (при переполнении знакового диапазона).

SET_H (при переносе из 3-го в 4-й разряд).

SET_S (вычисляется как $N \oplus V$).

5) Запись результата (Write Back)

Запись в регистр: УУ формирует сигнал **WRITE_REG** совместно с адресом SEL_RD=15. Результат суммы (SUM) по шине данных записывается обратно в регистр **R15**, замещая его предыдущее содержимое. Регистр R16 не изменяется.

Запись флагов: Формируется сигнал **UPDATE_SREG**, который разрешает запись новых состояний флагов (Z, C, N, V, H, S) в соответствующие триггеры регистра **SREG**.

6) Завершение цикла

Все этапы выполняются за **1 такт**. По окончании такта внутренние шины освобождаются, сигналы сбрасываются, и процессор переходит к этапу 1 для следующей команды по обновленному адресу в СчК.

2.5 Расшифровка команд и их аргументов

Bfff – переводим hex в двоичную систему счисления:

1011.1111.1111.1111

В инструкции к МК, был найден шаблон команды OUT:

1011.1AAr.rrrr.AAAA

Где AAAAA=111111_2 = 63_{10} – номер регистра ввода-вывода(порта)

rrrrr = 11111_2 = 31_{10} – номер регистра общего назначения

Получаем итоговую команду **OUT 0x3F, R31**

В коде ассемблера написали команду, получили файл .lss и проверили, что всё сходится.

```
000028 bfff                     OUT 0x3F, R31
```

93e8 – переводим hex в двоичную систему счисления:

1001.0100.1110.1000

В инструкции к МК, был найден шаблон команды CLT(Очистка флага передачи):

1001.0100.1110.1000

Получаем итоговую команду **CLT**

В коде ассемблера написали команду, получили файл .lss и проверили, что всё сходится.

```
000028 94e8                     CLT
```

ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

1. Перечислите основные узлы микроконтроллера ATmega32 и укажите их назначение.

Микроконтроллер **ATmega32** — это полноценный компьютер на одном кристалле. Его основные узлы можно разделить на три группы: ядро, память и периферия.

Ядро (Центральный процессор):

1) **АЛУ (Арифметико-Логическое Устройство):** Выполняет все математические операции (сложение, вычитание) и логические действия (И, ИЛИ, сдвиги). В AVR оно очень быстрое и работает за 1 такт.

2) **32 Регистра общего назначения (РОН):** Быстрая память внутри процессора для хранения промежуточных данных. Все регистры напрямую подключены к АЛУ, что позволяет обрабатывать два числа одновременно за один такт.

3) **Счетчик команд (PC):** Указывает адрес следующей команды в памяти программ, которую нужно выполнить.

4) **Регистр флагов (SREG):** Хранит результаты последних операций (был ли ноль, был ли перенос, отрицательное ли число и т.д.). Эти флаги нужны для условных переходов.

5) **Указатель стека (SP):** Указывает на вершину стека в оперативной памяти (нужен для подпрограмм и прерываний).

Память:

1) **Flash-память (ПЗУП):** Энергонезависимая память (32 Кбайта) для хранения самой программы. Она не стирается при выключении питания.

2) **SRAM (ОЗУ):** Оперативная память (2 Кбайта) для переменных во время работы программы. Стирается при отключении питания. Сюда же относятся регистры общего назначения.

3) **EEPROM**: Небольшая энергонезависимая память (1024 байта) для хранения важных данных (настроек), которые нужно сохранить после выключения, но которые могут меняться в процессе работы.

Периферия (Внешние устройства):

1) **Порты ввода-вывода (PORTA, PORTB, PORTC, PORTD)**: 32 ножки микросхемы для подключения светодиодов, кнопок, двигателей и других внешних устройств.

2) **Таймеры-счетчики:** Устройства для точного отсчета времени, генерации сигналов (ШИМ) или подсчета внешних импульсов.

3) **АЦП (Аналого-цифровой преобразователь):** Преобразует аналоговое напряжение (например, с датчика температуры) в цифровое число, которое понимает процессор.

4) **Интерфейсы связи (USART, SPI, I2C):** Позволяют общаться с другими микросхемами, компьютером или датчиками по проводам.

5) **Сторожевой таймер (Watchdog):** Сбрасывает микроконтроллер, если программа «зависла», чтобы система могла перезапуститься самостоятельно.

2. Укажите, в чём проявляются признаки RISC-архитектуры в микроконтроллере ATmega32. В чём преимущества и недостатки приведённых особенностей?

RISC (Reduced Instruction Set Computer) — архитектура с сокращенным набором команд.

Признаки RISC в ATmega32:

1) Фиксированная длина команд: Большинство команд занимают ровно 16 бит (одно машинное слово). Это упрощает декодирование.

2) Выполнение за 1 такт: Подавляющее большинство инструкций (более 120 из 131) выполняются за один такт процессора.

3) Регистровая архитектура: Все 32 регистра общего назначения напрямую подключены к АЛУ. Операции выполняются между регистрами, а не с памятью (как в CISC).

4) Раздельная память (Гарвардская архитектура): Память программ и память данных имеют разные шины. Процессор может одновременно читать команду из **Flash** и данные из **SRAM**.

5) Простые команды: Команды делают только одно простое действие (например, «сложи два регистра»), сложные действия разбиваются на последовательность простых.

Преимущества:

1) Высокая скорость: Благодаря выполнению за 1 такт и конвейерной обработке (пока выполняется одна команда, следующая уже считывается), производительность достигает до 1 MIPS на каждый МГц частоты (16 МГц = 16 млн операций в секунду).

2) Предсказуемость: Легко рассчитать точное время выполнения программы, что критично для систем реального времени.

3) Энергоэффективность: Простая схема потребляет меньше энергии

Недостатки:

Больший размер кода: Чтобы сделать сложное действие (например, сложить число из памяти с константой), нужно несколько команд (загрузить в регистр -> сложить -> записать обратно). Это занимает больше места в памяти программ по сравнению с CISC.

Сложность программирования на низком уровне: Программисту приходится писать больше строк кода на ассемблере для решения элементарных задач.

3. От чего зависит время выполнения команд CPSE и BRHS?

Время выполнения команд **CPSE** и **BRHS** зависит от **выполнения условия перехода/пропуска** и, в случае команды CPSE, от **длины следующей команды**.

1. Команда CPSE (Compare and Skip if Equal)

Назначение: Сравнение регистров, пропуск следующей команды, если они равны (if(Rd= Rr)).

Время выполнения (такты): 1/2/3 такта.

От чего зависит:

Результат сравнения: Если содержимое регистров не равно (условие ложно), команда выполняется за 1 такт, и следующая команда выполняется обычно.

```
.org 0x0000
    LDI R16, 10        ; 1. Загрузка константы 10
    LDI R17, 20        ; 2. Загрузка константы 20
    CPSE R16, R17       ; 3. Сравнение (10 != 20). Условие ложно.
(1 такт)
    NOP                ; 4. Команда выполняется (не пропускается)
    RET                ; 5. Возврат
```

Длина следующей команды (если условие истинно): Если содержимое регистров равно (условие истинно), следующая команда пропускается. Время выполнения зависит от длины пропускаемой команды:

2 такта, если следующая команда имеет длину 1 слово.

3 такта, если следующая команда имеет длину 2 слова.

```
.org 0x0000
    LDI R16, 5         ; 1. Загрузка константы 5
    LDI R17, 5         ; 2. Загрузка константы 5
    CPSE R16, R17       ; 3. Сравнение (5 == 5). Условие истинно.
(2 такта)
    NOP                ; 4. Команда пропускается (1 слово)
    RET                ; 5. Возврат
```

*(**Примечание:** Если бы вместо NOP стояла команда длиной 2 слова, например JMP, время выполнения CPSE составило бы 3 такта).*

2) Команда BRHS (Branch if Half Carry Set)

Назначение: Условный переход, если установлен флаг полу-переноса (if(H= 1)).

Время выполнения (такты): 1/2 такта.

От чего зависит:

Состояние флага H (Half Carry) в регистре SREG:

1 такт, если флаг **H = 0** (условие ложно, переход не выполняется).

```
.org 0x0000
    LDI R16, 0x01      ; 1. Загрузка 0x01
    LDI R17, 0x01      ; 2. Загрузка 0x01
    ADD R16, R17        ; 3. Сложение (0x02). Переноса из 3-го
бита нет (H=0)
    BRHS Skip              ; 4. Проверка флага H. Переход не
выполняется. (1 такт)
    NOP                ; 5. Команда выполняется
Skip:
    RET                ; 6. Возврат
```

2 такта, если флаг **H = 1** (условие истинно, переход выполняется, счетчик команд PC изменяется).

```
.org 0x0000
    LDI R16, 0x0F      ; 1. Загрузка 0x0F (0000 1111)
    LDI R17, 0x01      ; 2. Загрузка 0x01 (0000 0001)
    ADD R16, R17       ; 3. Сложение (0x10). Есть перенос из 3-го
бита (H=1)
    BRHS Skip              ; 4. Проверка флага H. Переход
выполняется. (2 такта)
    NOP                ; 5. Команда пропускается
Skip:
    RET                ; 6. Возврат
```

4. Приведите пример выполнения циклического сдвига вправо 8-разрядного числа (с переходом младшего разряда в старший) без использования флага T. Приведите пример из трёх машинных команд, обеспечивающих сложение 24-разрядного числа с 24-разрядной константой.

Для выполнения циклического сдвига вправо (младший бит переходит в старший) без использования флага T (команды BST/BLD), можно использовать флаг переноса C и команду условного перехода.

Логика:

1) Команда LSR сдвигает регистр вправо, при этом младший бит (Bit 0) попадает в флаг переноса C, а старший бит (Bit 7) очищается в 0.

2) Если флаг C равен 1 (значит, бывший младший бит был единицей), необходимо установить старший бит регистра в 1.

3) Если флаг C равен 0, старший бит остается 0 (что уже выполнено командой LSR).

```
LSR  R16          ; Логический сдвиг вправо. C <- R16(0), R16(7) <- 0
BRCC Skip         ; Если C=0 (бит был 0), переход к метке Skip
ORI  R16, 0x80    ; Если C=1 (бит был 1), устанавливаем старший бит (0x80 = 1000 0000)
Skip:
```

2) Сложение 24-разрядного числа с 24-разрядной константой (3 команды)

В системе команд ATmega32 отсутствует команда сложения с непосредственным операндом (ADI) для одиночных регистров. Однако есть команды вычитания с константой (SUBI, SBCI). Сложение можно реализовать через вычитание отрицательного числа (дополнительный код).

Условие:

1) Число хранится в регистрах **R16:R17:R18** (младший : средний : старший).

2) Регистры должны быть из диапазона **R16–R31** (требование команд SUBI/SBCI).

3) Константа **K** представляется в 24-разрядном дополнительном коде (отрицательное значение -K).

```
SUBI R16, LOW(-K)     ; Сложение младшего байта (через вычитание дополнения)
SBCI R17, MID(-K)     ; Сложение среднего байта с учётом переноса
SBCI R18, HIGH(-K)    ; Сложение старшего байта с учётом переноса
```

Пояснение:

SUBI выполняет Rd ← Rd - K. Если подать на вход -K (дополнительный код), операция превращается в Rd - (-K) = Rd + K.

SBCI выполняет Rd ← Rd - K - C. Аналогично работает для старших байтов с учетом переноса от младших.

Пример: Чтобы прибавить **1** (0x000001), нужно вычесть **-1** (0xFFFFFF).

5. Обоснуйте, чем вызваны ограничения допустимых значений номеров регистров и диапазонов констант в некоторых командах?

Ограничения вызваны **физической структурой машинного кода (размером инструкции)**.

Ограниченная длина команды: Большинство команд в AVR занимают всего **16 бит** (2 байта).

Структура слова: Из этих 16 бит часть обязательно уходит на **код операции (Opcode)** — чтобы процессор понял, что это за команда (например, ADD, LDI, JMP). На это обычно уходит от 4 до 8 бит.

Оставшееся место для операндов: То, что осталось (например, 8-10 бит), нужно разделить между номерами регистров и значениями констант.

Чтобы закодировать любой из 32 регистров, нужно **5 бит** ($2^5 = 32$).

Если в команде два регистра, нужно уже **10 бит**. Плюс код операции. Места в 16 битах едва хватает.

Если нужна еще и константа, места становится критически мало.

Примеры ограничений:

Команда LDI (загрузка константы): Ей нужно хранить саму константу (8 бит) и номер регистра. Чтобы уместить всё в 16 бит, разработчики пожертвовали диапазоном регистров. Поэтому LDI работает только с регистрами **R16–R31** (старшая половина). Для них достаточно закодировать только младшие 4 бита номера (так как старший бит всегда 1), что экономит место для константы.

Команды перехода (RJMP, BRNE): Им нужно хранить смещение (k). Чем больше бит выделено под смещение, тем дальше можно прыгнуть. В RJMP под смещение дано 12 бит (прыжок ±2К слов), а в коротких условных переходах (BRNE) — всего 7 бит (прыжок ±64 слова), потому что нужно было оставить место для кода условия.

Команды работы с парами регистров (ADIW, SBIW): Работают только с четырьмя конкретными парами (R24:R25, R26:R27, R28:R29, R30:R31), так как для кодирования выбора пары нужно очень мало бит (всего 2 бита), что позволяет вместить константу до 63.

Вывод: Ограничения — это компромисс между желанием сделать команду короткой (для быстродействия и экономии памяти) и необходимостью закодировать в ней все нужные данные. Если бы места было больше, команды стали бы длиннее (32 бита), что замедлило бы их выполнение и увеличило размер программы.

ВЫВОД

В ходе выполнения лабораторной работы №1 были изучены основы программирования микроконтроллеров семейства Atmel AVR в среде разработки Atmel Studio, рассмотрена архитектура микроконтроллера ATmega32, включая гарвардскую организацию памяти, систему регистров общего назначения и периферийные модули. Были освоены принципы работы с системой команд микроконтроллера, изучены форматы файлов .lss и .hex, а также получен практический опыт написания ассемблерного кода для реализации циклического сдвига 32-разрядных данных и вывода их на порты ввода-вывода. В практической части работы был выполнен расчёт времени выполнения подпрограммы задержки с учётом тактовой частоты процессора, исследован порядок выполнения команд на уровне ЦП, а также проведена расшифровка машинных кодов инструкций. Полученные знания позволяют понять принципы низкоуровневого программирования микроконтроллеров, особенности работы с регистрами, флагами состояния и периферией, что является важной основой для дальнейшего изучения аппаратных средств вычислительной техники и разработки встраиваемых систем.

Листинг «.asm файл»

```
.def TMP = R20 ; Создаём псевдоним TMP для регистра R20
.org $000      ; Следующая команда в самом начале памяти команд
JMP reset      ; Указатель на начало программы

; Функция паузы. два вложенных цикла, которые просто сжигают такты:
delay:
LDI R30, 200; y = 200 1 такт
LDI R29, 153; x = 153 1 такт

delay_sub:
INC R29           ; x++ (инкремент) 1 такт
NOP               ; 1 такт
BRNE delay_sub     ; «перейди по метке, если флаг Z = 0», т.е пока x!=0 (2 такта), если x=0 (1 такт)
NOP               ; 1 такт
DEC R30           ; y-- 1 такт
BRNE delay_sub     ; «перейди по метке, если флаг Z = 0», т.е пока y!=0 (2 такта), если y=0 (1 такт)
RET

; Начальная настройка
reset:

; настройка исходных значений
; LDI - LoaD Immediate (Загрузить непосредственно).
LDI TMP, 0x01    ; Загружаем шестнадцатеричное значение 0x01 (1 в десятичной) в TMP
MOV R0, TMP     ; Копируем значение из регистра TMP в регистр R0
CLR TMP         ; Устанавливает значение регистра TMP в 0
MOV R1, TMP     ; Копируем 0 из TMP в R1
MOV R2, TMP     ; Копирует 0 из TMP в R2
MOV R3, TMP     ; Копирует 0 из TMP в R3
```

```
; настройка портов ввода-вывода
; SER - (Set Register) Устанавливает все биты регистра в 1
SER TMP ; 0xFF
OUT DDRA, TMP ; Вывод (регистр направления данных (Data Direction Register))
OUT DDRB, TMP ; Вывод
OUT DDRC, TMP ; Вывод
OUT DDRD, TMP ; Вывод

; Запись 0xFF (все биты 1) все 8 контактов всех портов A, B, C, D настраиваются как выходы
; чтоб микроконтроллер мог выводить на них значения

; Установка вершины стека в конец ОЗУ
LDI TMP, HIGH(RAMEND) ; Старшие разряды адреса (8 бит максимального адреса)
OUT SPH, TMP          ; SPH - Stack Pointer High
LDI TMP, LOW(RAMEND)  ; Младшие разряды адреса
OUT SPL, TMP          ; SPL - Stack Pointer Low

; Основной цикл
loop:
; Циклический сдвиг 32-разрядного числа R0-R3
BST R0, 0 ; сохранение младшего бита во флаге T (т.е 1 у нас тут)
LSR R3 ; логический сдвиг вправо
ROR R2 ; циклический сдвиг вправо
ROR R1 ; циклический сдвиг вправо
ROR R0 ; циклический сдвиг вправо
BLD R3, 7 ; заполнение 7 бита значением из флага T

; Вывод 32-разрядного числа R0-R3 на порты PORTA-PORTD
OUT PORTA, R0
OUT PORTB, R1
OUT PORTC, R2
```

```
OUT PORTD, R3

; Пауза
CALL delay;

; Возврат в начало основного цикла
RJMP loop ;
```